

Diagnostic Médical

Projet : Système multi-agents médical avec LangGraph

Technologies cibles : LangGraph, LangChain, FastAPI, MCP,
React/Angular/Flutter/Streamlit

Pr. Mohamed YOUSSEFI

Technologies cibles : LangGraph, LangChain, FastAPI, MCP, React/Angular/Flutter/Streamlit

1. Contexte

Dans ce projet, les étudiants doivent concevoir une application multi-agents basée sur LangGraph pour simuler un workflow d'orientation clinique. Le système devra permettre de recueillir des informations patient, de produire une synthèse clinique préliminaire, d'intégrer une validation humaine par un médecin traitant, puis de générer un rapport final structuré.

Le projet devra ensuite être testé dans LangGraph Studio, exposé via une API FastAPI, enrichi avec une intégration MCP, et connecté à un frontend au choix.

2. Cadre pédagogique et éthique

- Le projet est un exercice académique.
- Le système ne doit pas être présenté comme un dispositif médical.
- Le système ne doit pas fournir de diagnostic définitif.
- Les termes recommandés sont : orientation clinique préliminaire, synthèse clinique, recommandation intermédiaire.
- Le rapport final doit mentionner explicitement : « Ce système ne remplace pas une consultation médicale. »

3. Objectifs pédagogiques

- Modéliser un workflow multi-agents avec LangGraph.
- Gérer un état partagé entre plusieurs agents.
- Utiliser des tools et intégrer le Human-in-the-Loop.
- Exposer le graphe via une API FastAPI.
- Intégrer MCP pour les outils des agents.
- Développer une interface utilisateur.
- Tester et déboguer le graphe dans LangGraph Studio.

4. Description fonctionnelle attendue

4.1 Agents obligatoires

- Supervisor : orchestre le workflow et décide de l'étape suivante.
- Diagnostic Agent : pose 5 questions au patient, réalise une pré-analyse et produit une synthèse clinique préliminaire.
- Report Agent : génère le rapport final structuré.
- Physician Review : étape Human-in-the-Loop représentant le médecin traitant.

4.2 Human-in-the-Loop obligatoire

- Le workflow doit inclure une interruption pour le médecin traitant.
- Le médecin reçoit la synthèse clinique et la recommandation intermédiaire.
- Le médecin propose un traitement ou une conduite à tenir avant le rapport final.

4.3 Interaction patient

- Le Diagnostic Agent doit poser 5 questions successives au patient.
- Ces questions doivent être gérées via un tool.
- Les réponses patient doivent être intégrées dans l'état du graphe.

4.4 Recommandation intermédiaire

- Le système doit proposer une recommandation intermédiaire générale.
- Cette recommandation reste prudente et ne remplace pas l'avis du médecin.
- Elle peut inclure : repos, hydratation, surveillance, consultation rapide en cas d'aggravation.

5. Workflow minimal attendu

```
START
|
v
Supervisor
|
v
DiagnosticAgent
|
+--> Tool: ask_patient (boucle jusqu'à 5 questions)
|
+--> Tool: recommend_interim_care
|
v
Supervisor
|
v
PhysicianReview (Human-in-the-Loop)
|
v
Supervisor
|
v
ReportAgent
|
v
Supervisor
|
v
END
```

6. Exigences techniques

- Backend : Python, LangGraph, LangChain, modèle LLM compatible.
- API : FastAPI obligatoire.
- MCP : intégration obligatoire d'au moins un outil.
- Frontend : React, Angular, Flutter ou Streamlit.

7. Architecture logicielle recommandée

```
project/
├── backend/
│   ├── app/
│   │   ├── graph.py
│   │   ├── state.py
│   │   └── nodes/
│   │       ├── supervisor.py
│   │       ├── diagnostic_agent.py
│   │       ├── physician_review.py
│   │       └── report_agent.py
│   │   └── tools/
│   │       ├── patient_tools.py
│   │       ├── care_tools.py
│   │       └── mcp_client.py
│   └── api.py
├── langgraph.json
├── requirements.txt
├── mcp_server/
│   ├── server.py
│   └── data/
├── frontend/
│   └── ...
└── README.md
```

8. État partagé du graphe

Exemple de structure de state :

```
from typing import Annotated
from typing_extensions import TypedDict, Literal
from langgraph.graph.message import add_messages

class MedicalState(TypedDict, total=False):
    messages: Annotated[list, add_messages]
    next: Literal[
        "diagnostic_agent",
        "physician_review",
        "report_agent",
        "FINISH"
    ]
    question_count: int
    interim_care: str
    diagnostic_summary: str
    physician_treatment: str
    final_report: str
```

9. Intégration MCP

L'intégration d'au moins un outil via MCP est obligatoire.

10. API FastAPI obligatoire

- POST /sessions/start
- POST /consultation/start
- POST /consultation/resume
- GET /consultation/{thread_id}
- GET /consultation/{thread_id}/report

11. Frontend obligatoire

- Le frontend peut être développé avec React, Angular, Flutter ou Streamlit.
- Il doit permettre de démarrer une consultation.
- Il doit afficher les questions du patient.
- Il doit gérer la revue du médecin traitant.
- Il doit afficher le rapport final.

11.1 Écrans minimums

- Écran 1 : saisie du cas initial patient.
- Écran 2 : questions/réponses patient.
- Écran 3 : revue médecin avec traitement ou conduite à tenir.
- Écran 4 : rapport final.

12. Test obligatoire dans LangGraph Studio

- Le graphe doit être visualisable dans LangGraph Studio.
- Les étudiants doivent démontrer les transitions entre les nœuds.
- Les interruptions patient et médecin doivent être testées.
- Les états intermédiaires doivent être observables.

13. Jeux de tests attendus

- Cas 1 : syndrome respiratoire simple.
- Cas 2 : cas avec red flags.
- Cas 3 : cas bénin.
- Pour chaque scénario, vérifier : 5 questions, recommandation intermédiaire, revue médecin, rapport final.

14. Livrables attendus

- Code source complet : backend, API, MCP, frontend.
- README.md avec instructions d'installation et d'exécution.
- Rapport technique court expliquant l'architecture et les choix réalisés.

- Démonstration ou capture d'écran/vidéo.
- Tests dans LangGraph Studio.

15. Contraintes minimales

- Utiliser LangGraph.
- Contenir un Supervisor.
- Contenir au moins deux agents métiers.
- Intégrer au moins un Human-in-the-Loop.
- Utiliser au moins un tool via MCP.
- Exposer une API FastAPI.
- Disposer d'un frontend.
- Être testable dans LangGraph Studio.

16. Bonus possibles

- Sortie JSON structurée avec Pydantic.
- Gestion avancée des erreurs.
- Persistance en base de données.
- Export PDF du rapport.
- Historique des consultations.
- Docker / docker-compose.
- Tests unitaires et tests d'intégration.

17. Critères d'évaluation proposés

- Architecture et conception : /20
- LangGraph : /20
- Agents et tools : /20
- FastAPI : /15
- Frontend : /15
- Documentation et démonstration : /10

18. Conseils de réalisation

1. Commencer par un graphe simple : Supervisor -> DiagnosticAgent -> ReportAgent -> END.
2. Ajouter ensuite la boucle ask_patient.
3. Ajouter le Human-in-the-Loop médecin.
4. Ajouter la recommandation intermédiaire.
5. Développer l'API FastAPI.
6. Brancher le frontend.

7. Intégrer MCP.
8. Tester l'ensemble dans LangGraph Studio.

19. Résultat final attendu

9. Un utilisateur saisit un cas patient.
10. Le Diagnostic Agent pose 5 questions.
11. Le système produit une synthèse clinique préliminaire.
12. Une recommandation intermédiaire est générée.
13. Le médecin traitant intervient manuellement.
14. Le Report Agent produit le rapport final.
15. L'ensemble est accessible via API et frontend.
16. Le graphe est démontré dans LangGraph Studio.